



AutoBonusRunner

Complete Manual



Routing, jumping, recovery, modes, rewards, configuration, and diagnostics
Version 1.0.0 | Tashi

AutoBonusRunner is the Bonus Stage module in Tashi's Full Automation Suite. It controls sphere requirements, terrain routing, jump timing, wall-climb recovery, reward actions, and the native retry choice.

This manual covers public version 1.0.0, internal version V1.22, and configuration version 44. New users should begin with the User Guide.

Reference Chapters

- 1 Installation and Upgrading
- 2 Quick Start and Controls
- 3 Modes and Sphere Requirements
- 4 Routing, Jumping, and Wall Recovery
- 5 Completion, Rewards, and Retry
- 6 Configuration Reference
- 7 Logging and Run Statistics
- 8 Troubleshooting

Module Responsibilities

- AutoBonusRunner controls supported Bonus Stage routes, collection, rewards, and the native Second Wind choice.
- AutoAdventurer handles active gameplay, quests, dimension travel, Rage, movement abilities, events, and bosses.
- AutoProgression handles purchases, Ascension, craftables, materials, eggs, and account maintenance.
- AutoClimber handles Ascending Heights routes, enemies, and rewards.

The mods can be used independently or together.

Default Control and Mode

Setting	Default
Toggle key	U
Enabled on startup	Yes
Mode	Auto
Auto retry	No
Skip start slider	Yes
Debug logging	No

Design Priorities

- 1 Keep the player on a reachable landing route.
- 2 Collect enough Bonus Spheres to complete the active section.
- 3 Adapt jump distance to observed speed and physical feedback.
- 4 Recover from unexpected walls, landings, and missed predictions.

- 5 Continue normal routing until a real reward target is confirmed.
- 6 Keep diagnostic detail available without requiring it during normal use.

Installation and Upgrading

Idle Slayer Mod Manager

- 1 Install and initialize Idle Slayer Mod Manager.
- 2 Install Idle Slayer Mods Core.
- 3 Import AutoBonusRunner.zip without extracting it.
- 4 Enable AutoBonusRunner and launch Idle Slayer.

The ZIP contains the DLL and Mod Manager metadata.

Manual Installation

Place AutoBonusRunner.dll in:

```
%LOCALAPPDATA%\IdleSlayerModManager\ModLoader\Mods\
```

Do not place it directly in the Idle Slayer game folder. If the Mod Manager uses a custom location, use that installation's ModLoader/Mods directory.

Required Mod

AutoBonusRunner requires Idle Slayer Mods Core. MelonLoader and the required game interop files are normally initialized by Idle Slayer Mod Manager.

The release was tested with MelonLoader 0.7.3 Open-Beta and Idle Slayer Mods Core 1.3.2.

Input Compatibility

Do not enable another mod that controls jump press, hold, and release during a Bonus Stage. AutoJumpMod and similar mods can conflict even when both work correctly on their own.

AutoBonusRunner does not require AutoAdventurer for routing, bow fire, or completion Wind Dash.

Configuration Location

The first launch creates:

```
%LOCALAPPDATA%\IdleSlayerModManager\ModLoader\UserData\AutoBonusRunner.cfg
```

Existing preferences are migrated automatically. Configuration Version is an internal migration value and should not be edited manually.

Upgrading

- 1 Close Idle Slayer.
- 2 Replace or re-import the old version.
- 3 Keep the existing configuration unless release notes say otherwise.
- 4 Launch the game and confirm the initialization line contains the expected public and internal versions.

Remove duplicate outdated AutoBonusRunner DLLs outside the locations managed by Idle Slayer Mod Manager.

Quick Start and Controls

Start Playing

- 1 Install AutoBonusRunner and launch the game once.
- 2 Keep Mode = "Auto" for normal unattended use.
- 3 Enter Bonus Stage 1, 2, or 3.
- 4 AutoBonusRunner takes control when supported Bonus gameplay becomes active.

No focus is required; route input can continue while the game window is in the background.

Toggle Key

Press U to disable or re-enable automatic control. The game displays a notification confirming the new state.

The key is read from Toggle Key. An invalid Unity key name falls back to U and writes a warning.

Startup State

- Enabled On Startup = true: automatic control begins enabled.
- Enabled On Startup = false: press the toggle key before use.

Disabling control releases mod-owned jump input immediately. Detection, logging, and manual-jump observation remain active.

Start Slider

With Skip Start Slider = true, the mod waits approximately one second after the Bonus Stage start slider appears. It confirms the slider only if the same slider is still visible.

Recommended Profiles

Normal unattended play

```
Mode = "Auto"
"Auto Retry Enabled" = false
"Skip Start Slider" = true
```

Full-route testing

```
Mode = "Manual"
"Debug Mode" = true
```

Fast completion in every Bonus Stage

```
Mode = "Skip"
```

Modes and Sphere Requirements

Auto

Auto is the default mode.

- Ordinary Bonus Stages use an effective requirement of 1 Bonus Sphere.
- Spirit Boost Bonus Stages preserve the game's native requirement.

After the requirement is met, terrain routing continues until a real reward object is confirmed. Reducing the requirement does not intentionally stop the character at the current platform.

Manual

Manual preserves the game's native sphere requirement in ordinary and Spirit Boost runs. Choose it for full-route testing, maximum collection attempts, or normal Bonus Stage rules in every run.

The name describes requirement behavior; automatic movement remains enabled unless it is toggled off with U.

Skip

Skip uses an effective requirement of 1 Bonus Sphere in both ordinary and Spirit Boost runs.

Skip changes only the section requirement. AutoBonusRunner still uses the normal route and reward controllers rather than teleporting or forcing the stage to end.

Requirement Safety

The effective value is selected by a postfix on the game's native requirement method. The runtime verifies that exactly one owned patch is present.

If that verification fails, Auto and Skip fail safely to the native requirement. Missing data does not invent a requirement.

Section Boundaries

Requirement choice is made for each active Bonus Stage section. Section, respawn, and retry transitions reset route calibration without discarding stable native physics observations that remain valid.

Routing, Jumping, and Wall Recovery

Terrain Model

AutoBonusRunner scans live colliders and combines them with authored Bonus Stage topology where available. A surface is represented as a horizontal landing interval with:

- raw and safe left and right boundaries;
- top height;
- map-piece and section identity;
- nearby alternatives, walls, and hazards.

Live physical contact remains authoritative when a cached or authored prediction disagrees with the game.

Jump Planning

Jump height and flight time depend on how long jump input is held. Candidate plans evaluate:

- current horizontal and vertical velocity;
- input-to-takeoff delay;
- observed jump impulse and gravity;
- hold duration;
- predicted landing interval;
- edge margin and landing safety;
- sphere and speed-boost intersections;
- Spirit Boost speed transitions.

The controller can wait for a launch window, issue a timed jump, or retain a safe passive route when no jump is required.

Sphere Collection

Visible Bonus Spheres are objectives, not guaranteed pickups. The planner balances collection with landing safety and the remaining section requirement.

Some stable authored layouts use a typed objective signature to prevent a generic planner from replacing a proven collection route. These signatures identify terrain and sphere shape, not a single world coordinate.

Small Gaps and Height Changes

A gap that the player footprint and current movement can safely traverse may be treated as continuous terrain. Larger gaps require a verified landing or a wall route.

Downward terrain can be walked, jumped, or entered intentionally depending on the next safe support and collectible route.

Physical Wall Ownership

When the player touches a real forward wall, physical contact overrides a stale predicted landing. The shared wall executor can:

- 1 issue an attached bounce;

- 2 verify upward physics steps;
- 3 release after the planned hold;
- 4 continue an attached climb;
- 5 transfer to the wall top or next wall;
- 6 return control after a stable support is confirmed.

Retries are bounded. A rejected impulse is not counted as a successful climb.

Recovery

Recovery can activate when:

- a jump is not accepted by the game;
- actual velocity differs from the prediction;
- the player lands on an intermediate support;
- a new physical wall is contacted;
- a target becomes stale or disappears;
- pit descent is confirmed;
- a respawn begins with temporary acceleration.

Recovery warnings identify the failure domain. Many warnings describe a successful correction rather than a death.

Background Input

The mod uses its own jump press, hold, and release state while route control is active. It does not depend on a foreground mouse click, allowing supported Bonus Stage control while the game is unfocused.

Completion, Rewards, and Retry

Sphere Completion Is Not the Reward

Meeting the sphere requirement is diagnostic evidence, not permission to abandon terrain routing. The same movement planner continues until the runtime confirms a real reward target.

Typed Reward Target

Reward control requires the same active reward box, coin, or gem to qualify on two consecutive observations. A latched target remains authoritative until an explicit stage reset.

Native reward flags and missing terrain are not enough by themselves to authorize reward actions.

Reward Actions

After the reward target is latched, the controller can:

- issue minimum jump pulses while grounded and stable;
- fire the bow at a controlled interval;
- activate the selected Wind Dash when its icon is visible, the ability is unlocked and ready, and the player is grounded or stably at ground height.

Wind Dash support is independent of AutoAdventurer.

Completion Traversal

The game can temporarily report that active collection has ended before the next section or reward is physically reached. AutoBonusRunner preserves route continuity during this transition and records completion-traversal state separately from native reward state.

Native Second Wind

AutoBonusRunner handles only the real one-use retry choice offered by the game:

- `Auto Retry Enabled = true`: choose Continue.
- `Auto Retry Enabled = false`: choose No and exit.
- automatic control disabled: leave the prompt for manual input.

Successful Continue consumption is never reset or recreated. Only a failed UI dispatch may be retried, with a bounded attempt count.

Configuration Reference

The configuration file is created after the first game launch:

```
%LOCALAPPDATA%\IdleSlayerModManager\ModLoader\UserData\AutoBonusRunner.cfg
```

AutoBonusRunner Section

Setting	Default	Description
Configuration Version	Managed	Internal preference migration version. Do not edit.
Debug Mode	false	Enables detailed route, input, physics, wall, landing, and completion diagnostics.

User actions, warnings, errors, and run summaries remain logged when Debug Mode is disabled.

AutoBonusRunner Automation Section

Setting	Default	Description
Mode	Auto	Selects Auto, Manual, or Skip requirement behavior.
Enabled On Startup	true	Starts automatic Bonus Stage control enabled.
Toggle Key	U	Keyboard key used to disable or re-enable automatic control.
Auto Retry Enabled	false	Chooses native Continue when true or No when false.
Skip Start Slider	true	Waits one second, then confirms the same visible start slider.

Automatic jumping, reward actions, and completion Wind Dash are built-in behaviors and are not separate configuration switches.

Default File

```
[AutoBonusRunner]
"Configuration Version" = 44
"Debug Mode" = false

[AutoBonusRunner Automation]
Mode = "Auto"
"Enabled On Startup" = true
"Toggle Key" = "U"
"Auto Retry Enabled" = false
"Skip Start Slider" = true
```

Editing Rules

- Restart the game after editing so every value is loaded consistently.
- Valid modes are Auto, Manual, and Skip; matching is case-insensitive.

- Use Unity key names such as U, F8, or Keypad1 for Toggle Key.
- Invalid keys fall back to U and produce a warning.
- Retired automatic-jump, reward-action, and Wind Dash entries are deleted during configuration migration.

Logging and Run Statistics

User Logs

Important lifecycle messages remain available with Debug Mode disabled:

- initialization and selected mode;
- enable or disable actions;
- retry decisions;
- section transitions;
- run results and aggregate statistics.

Errors remain visible because they can identify a real loading or runtime failure. Route, physics, landing, and wall-recovery warnings are diagnostic evidence and are emitted only when Debug Mode is enabled.

Independent Session Logs

AutoBonusRunner also writes a dedicated session trace:

```
%LOCALAPPDATA%\IdleSlayerModManager\ModLoader\UserData\AutoBonusRunner\Logs\
```

The filename contains the date, start time, and internal version. A new game session creates a new file, which makes it easier to avoid mixing tests from different DLLs.

Run Summary

A completed run prints fields such as:

```
Run count: Total=12, Success=12, Failure=0, PassRate=100.0%, Deathless=2, SessionDeaths=18, Deaths=1, Attempts=2, SectionsCleared=4/4, SpiritBoost=True.
```

Success describes complete Bonus Stage runs. Deaths and Attempts describe the current run, so a successful run can still include a death and a native retry.

Debug Mode

Enable Debug Mode for a reproducible route problem. It adds:

- diagnostic warnings;
- terrain and map-piece identity;
- current and alternative landing intervals;
- route candidates and rejection evidence;
- predicted speed, flight time, and landing;
- jump press, hold, release, and physics-frame evidence;
- wall contact and climb phases;
- sphere and boost progress;
- predicted-versus-actual landing results;
- completion target and reward actions.

Useful Problem Report

Include:

- 1 the latest complete independent log;
- 2 Bonus Stage 1, 2, or 3;
- 3 ordinary or Spirit Boost mode;
- 4 the section where the problem occurred;
- 5 whether any manual input was used;
- 6 a screenshot or video when the visual route is important.

Do not use a newly created startup-only log in place of the previous complete session.

Troubleshooting

AutoBonusRunner Does Nothing

- Press the configured toggle key and confirm the enabled notification.
- Confirm `Enabled On Startup = true` if no manual toggle is expected.
- Verify that AutoBonusRunner and Idle Slayer Mods Core are enabled.
- Confirm the active map is Bonus Stage 1, 2, or 3.
- Check the startup log for the expected internal version.

The Character Will Not Jump

- Disable AutoJumpMod and any other mod controlling jump input.
- Check that U has not disabled automatic control.
- Preserve warnings about input not being accepted by the game.
- Restart the game after replacing the DLL.

Jump Distance Is Wrong

Jump distance depends on current horizontal speed, Spirit Boost, press time, input delay, and collisions. A useful log must include the full approach, takeoff, and landing rather than only the death line.

Look for predicted and actual speed, flight time, landing position, and trajectory compatibility in Debug Mode.

The Character Stops at a Wall

Wall climbing uses bounded phases and requires a real physical contact. A rejected impulse can be retried, but repeated rejection at the same face may indicate stale wall ownership or an input barrier problem.

Preserve `WallClimbImpulseRejected`, `ReactiveWallRouteRebased`, and the surrounding physics-frame evidence.

The Sphere Requirement Looks Wrong

- Auto uses 1 sphere only in ordinary runs.
- Manual always preserves the native requirement.
- Skip always uses 1 sphere.
- Restart after changing Mode.
- Check startup patch inventory; missing verification falls back to the native value.

The Character Dies After Meeting the Requirement

The mod should continue normal routing until a real reward target is confirmed. Repeated deaths after `Remaining=0` are completion-traversal problems, not successful reward detection. Include the entire section log.

Retry Is Repeated or Does Nothing

AutoBonusRunner uses only the native one-use Second Wind choice. Set `Auto Retry Enabled` to true for Continue or false for No. If control is disabled, the prompt remains manual.

Include retry state, prompt callbacks, and UI-dispatch warnings when reporting a problem.

Background Control Does Not Work

AutoBonusRunner uses a background-compatible jump path only during supported route control. Check that another input mod, overlay, or game process setting is not blocking input.

Performance Drops

Static map and object data are cached. Heavy route planning should be bounded and event-driven. Enable Debug Mode for one reproducible run and look for performance, budget, or cache-reset evidence rather than leaving detailed logging enabled indefinitely.

IL2CPP Registration Errors

Use the newest DLL and remove duplicate outdated copies. Include the startup section of the MelonLoader log if registration still fails.